

# The types of dyadic interaction

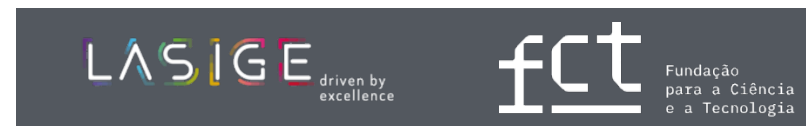
A tribute to Kohei Honda

by Diana Costa, Andreia Mordido, Diogo Poças, and Vasco T.

Vasconcelos

University of Lisbon

IFIP WG2.8 Viana do Castelo March 2026



This work is funded by national funds through FCT – Fundação para a Ciência e a Tecnologia, I.P. under the LASIGE Research Unit, ref. UID/00408/2025, DOI <https://doi.org/10.54499/UID/00408/2025>.

# How it all started

## Types for Dyadic Interaction

Kohei Honda

*kohei@mt.cs.keio.ac.jp*

Department of Computer Science, Keio University  
3-14-1 Hiyoshi, Kohoku-ku, Yokohama, 223, Japan

### Abstract

We formulate a typed formalism for concurrency where **types denote freely composable structure of dyadic interaction** in the symmetric scheme. The resulting calculus is a typed reconstruction of name passing process calculi. Systems with both the explicit and implicit typing disciplines, where types form a simple hierarchy of types, are presented, which are proved to be in accordance with each other. A typed variant of bisimilarity is formulated and it is shown that typed  $\beta$ -equality has a clean embedding in the bisimilarity. Name reference structure induced by the simple hierarchy of types is studied, which fully characterises the typable terms in the set of untyped terms. It turns out that the name reference structure results in the deadlock-free property for a subset of terms with a certain regular structure, showing behavioural significance of the simple type discipline.

Concur 1993

# The FreeST programming language



**Freest**

A functional programming language for safe concurrency  
powered by context-free session types

- Functional (call by value)
- Concurrent
- Communication by message passing (only)
- Channels are buffered, linear and governed by session types
- System  $F_{\omega}^{\mu}$

# 1 \_ Programming dyadic interaction

# The types of dyadic interaction

| Blocking                               | Input           | Output          | Non blocking    | Chain operations on values of these types |
|----------------------------------------|-----------------|-----------------|-----------------|-------------------------------------------|
|                                        | <b>Negative</b> | <b>Positive</b> |                 |                                           |
| Pattern match on values of these types | ?               | !               | Value exchange  |                                           |
|                                        | &               | $\oplus$        | Choice          |                                           |
|                                        | $\forall$       | $\exists$       | Type exchange   |                                           |
|                                        | Wait            | Close           | Channel closing |                                           |

# Infinitely repeating some action a

Multiplicity  
(linear)

Kind  
(Session)

```
type IRepeat : 1S -> 1S
```

```
type IRepeat a = a ; IRepeat a
```

# Infinite streams of some type *a*

Multiplicity  
(unrestricted)

Kind  
(Type/any)

```
type IStream, CoIStream : *T -> 1S
```

```
type IStream a = IRepeat (?a)      -- seen from the reader
```

```
type CoIStream a = Dual (IStream a) -- seen from the writer
```

A  
(internalised) type  
operator

# A consumer of type `IStream Int`

Pattern  
matching on negative  
type constructor

A type operator for  
non-inhabited types

```
echo : IStream Int -> Void @*T
```

```
echo (?x ; c) = printInt x ; echo c
```

Input x

Continue  
as c

```
type IStream Int ≈ ?Int ; ?Int ; ...
```



# A consumer of type CoIStream

Reverse function  
application  
 $x \mid> f = f \ x$

Chaining  
operations on  
positive types

```
ints :: Int -> CoIStream Int -> Void @*T
ints n c =
  c |> send n |> ints (n + 1)      -- preferred
  ints (n + 1) (send n c)          -- the functional way
  let c = send n c in ints (n + 1) c -- alternative
```

```
type CoIStream Int ≈ !Int ; !Int ; ...
```

# Finite or infinite streams

```
type IRepeat : 1S -> 1S -> 1S
type IRepeat a b = IRepeat (&{More: a, Done: b ; Wait})           -- unfold
-- ≈ &{More: a, Done: b ; Wait}; IRepeat a b                        -- distributivity
-- ≈ &{More: a ; IRepeat a b, Done: b ; Wait ; IRepeat a b}      -- Wait is absorbing
-- ≈ &{More: a ; IRepeat a b, Done: b ; Wait}                     -- fold
-- ≈ μc.&{More: a ; c, Done: b ; Wait}
```

# A consumer of type `IFRepeat (?a) Skip`

```
length : IFRepeat (?a) Skip -> Int
```

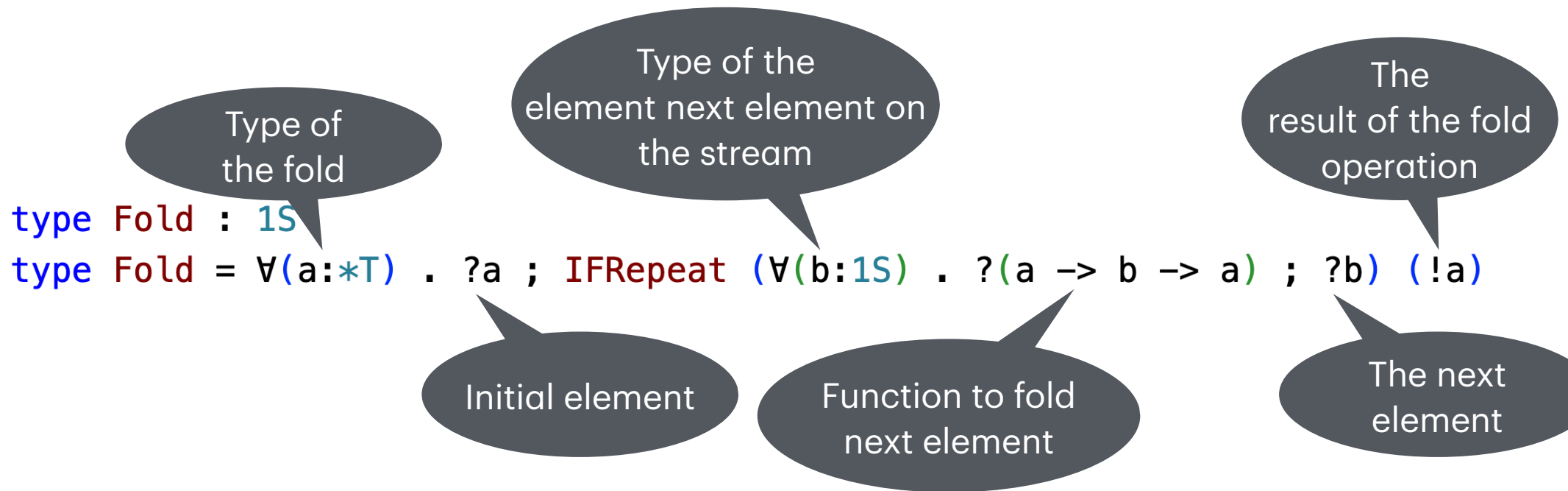
```
length (&Done Wait) = 0
```

```
length (&More (?_ ; c)) = 1 + length c
```

Nested pattern  
matching on negative type  
constructors

```
type IFRepeat (?a) Skip  $\approx$   $\mu c.$ &{More: ?a ; c, Done: Wait}
```

# Folding a stream of heterogeneous values



# A consumer for type Dual Fold

Expect return value  
"5True"

```
showStream : Dual Fold -> String
```

```
showStream c =
```

```
  let (x, c) = c |> sendType @String |> send ""
```

```
    |> select More |> sendType @Int
```

```
    |> send (\(x:String) (y:Int) -> x ++ showInt y) |> send 5
```

```
    |> select More |> sendType @Bool
```

```
    |> send (\(x:String) (y:Bool) -> x ++ showBool y) |> send True
```

```
    |> select Done
```

```
    |> receive
```

```
  in close c ; x
```

```
type Dual Fold ≈ ∃a . !a ; μc.+{More: ∃b . !(a -> b -> a) ; !b ; c, Done: ?a ; Close}
```

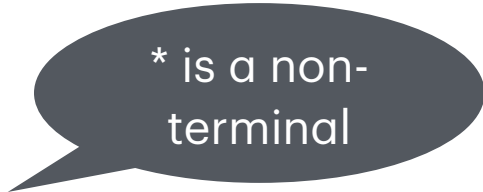
# A consumer of type Fold

```
fold : Fold -> ()  
fold (@a, (?x ; c)) = fold' x c  
  where  
    fold' x (&Done c) = c |> send x |> wait  
    fold' x (&More (@b, (?f ; ?y; c))) = fold' (f x y) c
```

```
type Fold ≈ ∀a . ?a ; μc.&{More: ∀b . ?(a -> b -> a) ; ?b ; c, Done: !a ; Wait}
```

2\_Type equivalence

# Kinds



\* is a non-terminal

\* :: =

T

S

$\kappa$  :: =

\*

$\kappa \Rightarrow \kappa$

Kind of proper types:

functional

session

Kind:

kind of proper types

kind of type operators



# The syntax of types

Cf.

Alonzo Church. 1940. A  
Formulation of the Simple Theory  
of Types

$T ::=$

$\iota$

$\alpha_{\kappa}$

$\lambda\alpha_{\kappa}.T$

$TT$

Type:

type constant

type variable

type-level abstraction

type-level application

# Type constants

$\# ::= ? \mid !$

$\langle \rangle ::= \{ \} \mid \langle \rangle$

$\iota ::=$

$\rightarrow_{* \Rightarrow *' \Rightarrow T}$

$(\bar{\ell})_{* \Rightarrow T}$

$\text{Msg}^{\#}_{* \Rightarrow S}$

$\#\{\bar{\ell}\}_{S \Rightarrow S}$

$;\text{S} \Rightarrow \text{S} \Rightarrow \text{S}$

$\text{End}^{\#}_S$

$\text{Skip}_S$

$\text{Dual}_{S \Rightarrow S}$

$\exists^{\#}_{(\kappa \Rightarrow *) \Rightarrow *}$

$\mu_{(\kappa \Rightarrow \kappa) \Rightarrow \kappa}$

$\text{Void}_{\kappa}$

Polarity:

Record, variant

Type constant:

arrow

record, variant

message (input, output)

choice (external, internal)

sequential composition

end (wait, close)

skip

dual operator

quantifier (universal, existential)

recursive type

void

# Some laws

- (1)  $(\lambda\alpha.T)U \simeq T[U/\alpha]$
- (2)  $\mu F \simeq F(\mu F)$
- (3)  $\mu(\lambda\alpha.T) \simeq T$  if  $\alpha \notin \text{fv}(T)$
- (4)  $\text{Skip}; T \simeq T$
- (5)  $\#\{\overline{l: T}\}; U \simeq \#\{\overline{l: T}; U\}$
- (6)  $(\exists^\# F); U \simeq \exists^\# \lambda\alpha.(F\alpha; U)$  if  $\alpha \notin \text{fv}(F, U)$
- (7)  $\text{Dual Skip} \simeq \text{Skip}$
- (8)  $\text{Dual End}^\# \simeq \text{End}^{\#\perp}$
- (9)  $\text{Dual Void} \simeq \text{Void}$
- (10)  $\text{Dual}(\text{Msg}^\# T) \simeq \text{Msg}^{\#\perp} T$
- (11)  $\text{Dual}(\#\{\overline{l: T}\}) \simeq \#\perp\{\overline{l: \text{Dual } T}\}$
- (12)  $\text{Dual}(T; U) \simeq \text{Dual } T; \text{Dual } U$
- (13)  $\text{Dual}(\text{Dual } T) \simeq T$
- (14)  $\text{Dual}(\exists^\# F) \simeq \exists^{\#\perp} \lambda\alpha.\text{Dual}(F\alpha)$  if  $\alpha \notin \text{fv}(F)$
- (15)  $\text{End}^\#; T \simeq \text{End}^\#$  and  $\text{Void}; T \simeq \text{Void}$
- (16)  $F \simeq \lambda\alpha.F\alpha$  if  $\alpha \notin \text{fv}(F)$
- (17)  $(T; U); V \simeq T; U; V$

Type equivalence is a weak  
bisimulation

# Bisimulation

- Introduced by Park in the early eighties, popularised by Milner (CCS, pi-calculus) as a behavioural equivalence for processes
- Used by Amadio and Cardelli in Subtyping recursive types (1993)

# Type equivalence is a weak bisimulation

- Includes internal actions:
  - Syntactically rearrange the type into a whnf, exposing a visible action
  - Aka type reduction
  - Not to be matched by the other type
- And external actions:
  - To be matched by the other type

# Some invisible actions (the tau rules)

Call by name and unfold

R- $\beta$

$$(\lambda\alpha.T)U \xrightarrow{\tau} T[U/\alpha]$$

R-APP L

$$\frac{F \xrightarrow{\tau} G}{FT \xrightarrow{\tau} GT}$$

R- $\mu$

$$\mu F \xrightarrow{\tau} F(\mu F)$$

# Some invisible actions (the tau rules)

The semicolon rules

$$\text{R-SNEUT} \quad \text{Skip}; T \xrightarrow{\tau} T$$

$$\text{R-SAssoc} \quad \frac{T = \text{End}^\#, \text{Void}, \text{Msg}^\# T_1, \alpha T_1 \cdots T_m, \text{Dual}(\alpha T_1 \cdots T_m)}{(T; U); V \xrightarrow{\tau} T; (U; V)}$$

$$\text{R-SCHOICEDIST} \quad \#\{\ell : \overline{T}\}; U \xrightarrow{\tau} \#\{\ell : \overline{T; U}\}$$

$$\text{R-SQUANTDIST} \quad \frac{\alpha \notin \text{fv}(F) \cup \text{fv}(U)}{(\exists^\# F); U \xrightarrow{\tau} \exists^\# \lambda \alpha. (F\alpha; U)}$$

$$\text{R-SSEMIL} \quad \frac{T \xrightarrow{\tau} V}{T; U \xrightarrow{\tau} V; U}$$



# Some action rules

## Fully applied type constructors

- Each fully applied type constructor of arity  $m$  shall have  $m+1$  transitions

A-CONST

$$\frac{\iota = \rightarrow, (\bar{\ell}), \text{Msg}^\#, \#\{\bar{\ell}\}, \text{End}^\#, \exists^\#}{(\iota T_1 \cdots T_m)_* \xrightarrow{l} \text{Skip}}$$

A-CONSTARG

$$\frac{\iota = \rightarrow, (\bar{\ell}), \text{Msg}^\#, \#\{\bar{\ell}\}, \exists^\#}{(\iota T_1 \cdots T_m)_* \xrightarrow{j} T_j}$$

$$m \geq 0 \text{ and } 1 \leq j \leq m$$

# Some action rules

## Higher-order types

- Higher-order types shall have an infinite number of transitions
- One for each variable of the appropriate kind

$$\text{A-ABS} \\ F_{\kappa \Rightarrow \kappa'} \xrightarrow{\lambda \alpha_{\kappa}} F_{\kappa \Rightarrow \kappa'} \alpha_{\kappa}$$

N.B.: Yields infinitely branching bisimulations

# Some action rules

What if reduction diverges?

- Then the type transits to Skip by a Void action

$$\frac{\text{A-DIVERGE} \quad T_* \text{ diverges}}{T_* \xrightarrow{\text{Void}_*} \text{Skip}}$$

The diverges predicate (coinductive)

$$\text{D-VOID} \quad \text{Void}_* \text{ diverges}$$

$$\frac{\text{D-APP} \quad F\alpha \text{ diverges}}{F \text{ diverges}}$$

$$\frac{\text{D-RED} \quad T_* \xrightarrow{\tau} U_* \quad U_* \text{ diverges}}{T_* \text{ diverges}}$$

# Example

Various internal reductions followed by external actions

- Take  $T \triangleq \mu(\lambda\beta.(\&\{\text{More} : (\forall\alpha.?\alpha), \text{Done} : \text{Wait}\}); \beta)$
- Then

$$T \xrightarrow{\tau} (\lambda\beta.(\&\{\text{More} : (\forall\alpha.?\alpha), \text{Done} : \text{Wait}\}); \beta) T \quad (\text{R-}\mu)$$

$$\xrightarrow{\tau} \&\{\text{More} : (\forall\alpha.?\alpha), \text{Done} : \text{Wait}\}; T \quad (\text{R-}\beta)$$

$$\xrightarrow{\tau} \&\{\text{More} : (\forall\alpha.?\alpha); T, \text{Done} : \text{Wait}; T\} \quad (\text{R-SCHOICEDIST})$$

↙  $\&\{\text{More}, \text{Done}\}$   
Skip

↓<sub>1</sub>  
 $(\forall\alpha.?\alpha); T$

↘<sub>2</sub>  
Wait; T

3\_Type equivalence

# Difficulty #1 \_ Dealing with infinitely branching bisims

- Recall A-Abs
- One transition per variable of kind  $k$

$$\text{A-Abs} \\ F_{\kappa \Rightarrow \kappa'} \xrightarrow{\lambda \alpha_{\kappa}} F_{\kappa \Rightarrow \kappa'} \alpha_{\kappa}$$

- **Solution:** we only need transitions by **two** (fixed) variables
- Fix two variables for each kind (vars that may appear in types)

$$\text{A-Abs1} \\ F_{\kappa \Rightarrow \kappa'} \xrightarrow{\lambda \alpha_{1\kappa}} F_{\kappa \Rightarrow \kappa'} \alpha_{1\kappa}$$

$$\text{A-Abs2} \\ F_{\kappa \Rightarrow \kappa'} \xrightarrow{\lambda \alpha_{2\kappa}} F_{\kappa \Rightarrow \kappa'} \alpha_{2\kappa}$$

- Show that  $T \simeq U$  iff  $T \simeq_f U$

# Difficulty #2 \_ Determining when a type diverges

- Recall

The diverges predicate (coinductive)

D-VOID  
 $\text{Void}_*$  diverges

D-APP  
 $\frac{F\alpha \text{ diverges}}{F \text{ diverges}}$

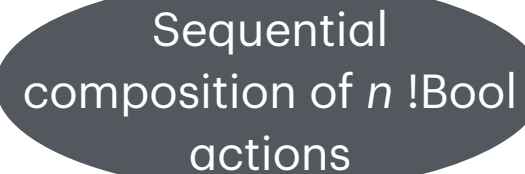
D-RED  
 $\frac{T_* \xrightarrow{\tau} U_* \quad U_* \text{ diverges}}{T_* \text{ diverges}}$

- Solution:** Restrict the recursion operator to base kinds  $\mu_{(T \Rightarrow T) \Rightarrow T}$  and  $\mu_{(s \Rightarrow s) \Rightarrow s}$
- Then divergence becomes tractable
- N.B.: Higher-order recursion is theoretically possible, its equivalence is tied to the bisimilarity of deterministic pushdown automata

## Difficulty #3 \_ Bisimulations may be infinite relations

- Take  $T \triangleq \mu\lambda\alpha.?\text{Int};\alpha$

$$U \triangleq \mu\lambda\beta.?\text{Int};\beta;!\text{Bool}$$



Sequential  
composition of  $n$  !Bool  
actions

- Then  $T \simeq U$  but the bisimulation is infinite:  $\{(T, U; !\text{Bool}^n), (\text{Int}, \text{Int}), (\text{Skip}, \text{Skip}) \mid n \geq 0\}$

- **Solution:**

- Translate types to (simple) grammars
- Run a (polynomial) algorithm to check whether two words in a simple grammar are bisimilar



FreeST 3.2  $F^\mu$ ;

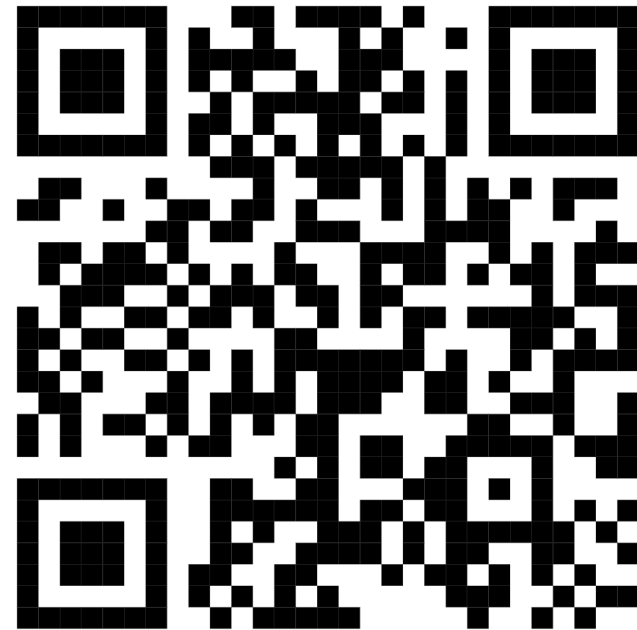
<https://freest-lang.github.io/>



Up and running

FreeST 5.0  $F_{\omega}^{\mu*}$ ;

<https://github.com/freest-lang/freest>



Coming soon

Extra material

# Embracing non-contractivity

- Recall `type IRepeat a = a ; IRepeat a`
- Then 
$$\begin{aligned}\text{IRepeat Skip} &\approx \mu\beta.\text{Skip}; \beta \\ &\approx \text{Skip}; \mu\beta.\text{Skip}; \beta. \\ &\approx \mu\beta.\text{Skip}; \beta\end{aligned}$$
- Instead of ruling out type `IRepeat` we deem `IRepeat Skip` well formed
- We further introduce a family of types `Voidk`, one for each kind `k`
- Then  $\text{IRepeat Skip} \approx \text{Void}_s$ :

# The full set of tau rules

$$\begin{array}{c}
 \text{R-}\beta \\
 (\lambda\alpha.T)U \xrightarrow{\tau} T[U/\alpha]
 \end{array}
 \quad
 \begin{array}{c}
 \text{R-}\mu \\
 \mu F \xrightarrow{\tau} F(\mu F)
 \end{array}
 \quad
 \begin{array}{c}
 \text{R-Void} \\
 \text{Void}_{\kappa \Rightarrow \kappa'} T \xrightarrow{\tau} \text{Void}_{\kappa'}
 \end{array}
 \quad
 \begin{array}{c}
 \text{R-APPL} \\
 \frac{F \xrightarrow{\tau} G}{FT \xrightarrow{\tau} GT}
 \end{array}
 \quad
 \begin{array}{c}
 \text{R-SNEUT} \\
 \text{Skip}; T \xrightarrow{\tau} T
 \end{array}$$
  

$$\begin{array}{c}
 \text{R-SAssoc} \\
 \frac{T = \text{End}^\#, \text{Void}, \text{Msg}^\# T_1, \alpha T_1 \dots T_m, \text{Dual}(\alpha T_1 \dots T_m)}{(T; U); V \xrightarrow{\tau} T; (U; V)}
 \end{array}
 \quad
 \begin{array}{c}
 \text{R-SCHOICEDIST} \\
 \#\{\ell : \overline{T}\}; U \xrightarrow{\tau} \#\{\ell : \overline{T; U}\}
 \end{array}$$
  

$$\begin{array}{c}
 \text{R-SQUANTDIST} \\
 \frac{\alpha \notin \text{fv}(F) \cup \text{fv}(U)}{(\exists^\# F); U \xrightarrow{\tau} \exists^\# \lambda\alpha.(F\alpha; U)}
 \end{array}
 \quad
 \begin{array}{c}
 \text{R-SSEMI} \\
 \frac{T \xrightarrow{\tau} V}{T; U \xrightarrow{\tau} V; U}
 \end{array}
 \quad
 \begin{array}{c}
 \text{R-DSKIP} \\
 \text{Dual Skip} \xrightarrow{\tau} \text{Skip}
 \end{array}
 \quad
 \begin{array}{c}
 \text{D-Void} \\
 \text{Dual Void} \xrightarrow{\tau} \text{Void}
 \end{array}$$
  

$$\begin{array}{c}
 \text{R-DEND} \\
 \text{Dual End}^\# \xrightarrow{\tau} \text{End}^{\#\perp}
 \end{array}
 \quad
 \begin{array}{c}
 \text{R-DMsg} \\
 \text{Dual}(\text{Msg}^\# T) \xrightarrow{\tau} \text{Msg}^{\#\perp} T
 \end{array}
 \quad
 \begin{array}{c}
 \text{R-DCHOICE} \\
 \text{Dual}(\#\{\ell : \overline{T}\}) \xrightarrow{\tau} \#\perp\{\ell : \overline{\text{Dual } T}\}
 \end{array}$$
  

$$\begin{array}{c}
 \text{R-DQUANT} \\
 \frac{\alpha \notin \text{fv}(F)}{\text{Dual}(\exists^\# F) \xrightarrow{\tau} \exists^{\#\perp} \lambda\alpha.\text{Dual}(F\alpha)}
 \end{array}
 \quad
 \begin{array}{c}
 \text{R-DSEMI} \\
 \text{Dual}(T; U) \xrightarrow{\tau} \text{Dual } T; \text{Dual } U
 \end{array}
 \quad
 \begin{array}{c}
 \text{R-DDUAL} \\
 \text{Dual}(\text{Dual } T) \xrightarrow{\tau} T
 \end{array}$$
  

$$\begin{array}{c}
 \text{R-DAPPR} \\
 \frac{T \neq T_1; T_2, \text{Dual } T_1 \quad T \xrightarrow{\tau} U}{\text{Dual } T \xrightarrow{\tau} \text{Dual } U}
 \end{array}$$

# The full set of action rules

$$\frac{\text{A-CONST} \quad \iota = \rightarrow, (\bar{\ell}), \text{Msg}^\#, \#\{\bar{\ell}\}, \text{End}^\#, \exists^\#}{(\iota T_1 \cdots T_m)_* \xrightarrow{\bar{\ell}} \text{Skip}}$$

$$\frac{\text{A-CONSTARG} \quad \iota = \rightarrow, (\bar{\ell}), \text{Msg}^\#, \#\{\bar{\ell}\}, \exists^\#}{(\iota T_1 \cdots T_m)_* \xrightarrow{j} T_j}$$

$$\text{A-VAR} \quad (\alpha T_1 \cdots T_m)_* \xrightarrow{\alpha} \text{Skip}$$

$$\text{A-VARARG} \quad (\alpha T_1 \cdots T_m)_* \xrightarrow{j} T_j$$

$$\text{A-DUAL} \quad \text{Dual}(\alpha T_1 \cdots T_m) \xrightarrow{\alpha^\perp} \text{Skip}$$

$$\text{A-DUALARG} \quad \text{Dual}(\alpha T_1 \cdots T_m) \xrightarrow{j} T_j$$

$$\text{A-ABS} \quad F_{\kappa \Rightarrow \kappa'} \xrightarrow{\lambda \alpha_\kappa} F_{\kappa \Rightarrow \kappa'} \alpha_\kappa$$

$$\frac{\text{A-DIVERGE} \quad T_* \text{ diverges}}{T_* \xrightarrow{\text{Void}_*} \text{Skip}}$$

$$\text{A-SVOID} \quad \text{Void}; T \xrightarrow{\text{Void}} \text{Skip}$$

$$\text{A-SEND} \quad \text{End}^\#; T \xrightarrow{\text{End}^\#} \text{Skip}$$

$$\text{A-SVAR} \quad (\alpha T_1 \cdots T_m); U \xrightarrow{\alpha} U$$

$$\text{A-SVARARG} \quad (\alpha T_1 \cdots T_m); U \xrightarrow{j} T_j$$

$$\text{A-SMSG} \quad \text{Msg}^\# T; U \xrightarrow{\text{Msg}^\#} U$$

$$\text{A-SMSGARG} \quad \text{Msg}^\# T; U \xrightarrow{1} T$$

$$\text{A-SDUAL} \quad (\text{Dual}(\alpha T_1 \cdots T_m)); U \xrightarrow{\alpha^\perp} U$$

$$\text{A-SDUALARG} \quad (\text{Dual}(\alpha T_1 \cdots T_m)); U \xrightarrow{j} T_j$$

Where  $m \geq 0$  and  $1 \leq j \leq m$  in all rules.

# Example of divergence I

$$T \triangleq \mu(\lambda\alpha_{s \Rightarrow s}.\alpha) : s \Rightarrow s$$

$$\frac{\frac{\frac{}{T \xrightarrow{\tau} (\lambda\alpha.\alpha)T} \text{R-}\mu}{(\lambda\alpha.\alpha)T \xrightarrow{\tau} T} \text{R-}\beta \quad \frac{\frac{}{T \text{ diverges}} \vdots}{(\lambda\alpha.\alpha)T \text{ diverges}} \text{D-RED}}{T \text{ diverges}} \text{D-RED}$$

# Example of divergence II

$\text{Void}_{s \Rightarrow s}$

$$\frac{\frac{\frac{}{\text{Void}_{s \Rightarrow s} \delta \xrightarrow{\tau} \text{Void}_s} \text{R-VOID} \quad \frac{\frac{}{\text{Void}_s \text{ diverges}} \text{D-VOID}}{\text{D-RED}}}{\frac{\text{Void}_{s \Rightarrow s} \delta \text{ diverges}}{\text{Void}_{s \Rightarrow s} \text{ diverges}} \text{D-APP}}$$

# Example of divergence III

$\lambda\gamma_s.\mu F : s \Rightarrow s$  where  $\hat{F} \triangleq \lambda\hat{\beta}_s.\beta;\gamma$ .

$$\begin{array}{c}
 \frac{}{(\lambda\gamma.\mu F)\gamma \xrightarrow{\tau} \mu F} \text{R-}\beta \quad \frac{}{\mu F \xrightarrow{\tau} F(\mu F)} \text{R-}\mu \quad \frac{\frac{}{\mu F \xrightarrow{\tau} F(\mu F)} \text{R-}\mu}{(\mu F);\gamma \xrightarrow{\tau} (F(\mu F));\gamma} \text{R-SSemIL} \quad \frac{\vdots \quad ((\mu F);\gamma);\gamma}{(F(\mu F));\gamma \text{ diverges}} \text{D-RED} \\
 \frac{}{(\lambda\gamma.\mu F)\gamma \xrightarrow{\tau} \mu F} \text{R-}\beta \quad \frac{\frac{}{\mu F \xrightarrow{\tau} F(\mu F)} \text{R-}\mu}{(\mu F);\gamma \text{ diverges}} \text{D-RED} \quad \frac{}{(\mu F);\gamma \text{ diverges}} \text{D-RED} \\
 \frac{}{(\lambda\gamma.\mu F)\gamma \text{ diverges}} \text{D-APP} \quad \frac{}{\lambda\gamma.\mu F \text{ diverges}} \text{D-APP}
 \end{array}$$



# Weak bisimulation

*Definition 4.3 (Weak bisimulation and weak bisimilarity).* A weak bisimulation is a relation  $R \subseteq F_{\omega}^{\mu_i} \times F_{\omega}^{\mu_i}$  such that, for every  $(T, U) \in R$  and every transition label  $a$ ,

- **(zig)** if  $T \xrightarrow{a} T'$ , then there exists  $U'$  such that  $U \xRightarrow{a} U'$  and  $(T', U') \in R$ ;
- **(zag)** if  $U \xrightarrow{a} U'$ , then there exists  $T'$  such that  $T \xRightarrow{a} T'$  and  $(T', U') \in R$ ;

Types  $T, U$  are said to be weakly bisimilar, denoted  $T \simeq U$ , if there exists a weak bisimulation  $R$  such that  $(T, U) \in R$ .

--

# Translation to grammar

$$\text{word}(\text{Skip}) = \varepsilon \quad (\text{W-SKIP})$$

$$\text{word}(\iota) \mid \iota = \text{End}^\#, \text{Void}_* = X \text{ with } X \xrightarrow{\iota} \perp \quad (\text{W-ENDVOID})$$

$$\text{word}(\text{Msg}^\# T) = X \text{ with } X \xrightarrow{\text{Msg}^\#} \varepsilon \text{ and } X \xrightarrow{1} \text{word}(T) \perp \quad (\text{W-MSG})$$

$$\text{word}(T; U) = \text{word}(T) \text{ word}(U) \quad (\text{W-SEQ})$$

$$\text{word}(\text{Dual}(\alpha T_1 \cdots T_m)) = X \text{ with } X \xrightarrow{\alpha^\perp} \varepsilon \text{ and } X \xrightarrow{j} \text{word}(T_j) \perp \quad (\text{W-DUALVAR})$$

$$\text{word}((\iota T_1 \cdots T_m)_*) \mid \iota = \rightarrow, (\bar{\ell}), \#\{\ell\}, \exists^\# = X \text{ with } X \xrightarrow{\iota} \perp \text{ and } X \xrightarrow{j} \text{word}(T_j) \quad (\text{W-CONST})$$

$$\text{word}((\alpha T_1 \cdots T_m)_*) = X \text{ with } X \xrightarrow{\alpha} \varepsilon \text{ and } X \xrightarrow{j} \text{word}(T_j) \perp \quad (\text{W-VAR})$$

$$\text{word}(T) \mid \mu\text{redex}(T) \text{ defined} \wedge T \Downarrow \text{Skip} = \varepsilon \quad (\text{W-}\mu\text{SKIP})$$

$$\text{word}(T) \mid \mu\text{redex}(T) \text{ defined} \wedge T \Downarrow U = X \text{ with } X \xrightarrow{a} \gamma\delta, \quad (\text{W-}\mu\text{NSKIP})$$

$$\text{where } Z\delta = \text{word}(U) \text{ and } Z \xrightarrow{a} \gamma$$

$$\text{word}(F_{\kappa \Rightarrow \kappa'}) = X \text{ with } X \xrightarrow{\lambda\alpha_\kappa} \text{word}(F\alpha_\kappa) \text{ and } X \xrightarrow{\lambda\beta_\kappa} \text{word}(F\beta_\kappa) \quad (\text{W-ABS})$$

$$\text{word}(T) \mid T \xrightarrow{\tau} U = \text{word}(U) \quad (\text{W-}\tau)$$